

**LAPORAN TUGAS BESAR
PEMROSESAN PARALEL *CLASS C* 2025**

HIERARCHICAL *CLUSTERING* ALGORITHM

**Implementasi Algoritma Hierarchical *Clustering* dengan
Pemrosesan Paralel untuk Klasterisasi Data Pelanggan Mall**



DISUSUN OLEH:

BAGUS ESA WIJAYA KUSUMA (F1D02310005)

VALERINE JESIKA DEWI (F1D02310027)

NAUFAL IHSANUL ISLAM (F1D02310084)

PUDAEL ZIKRI (F1D02310088)

M. DANUARTA WIRAGUNA (F1D02310124)

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MATARAM
2025**

BAB I

PENDAHULUAN

Dalam era digital, data pelanggan mall yang meliputi informasi seperti kebiasaan belanja, tingkat pengeluaran, dan latar belakang demografis, terus meningkat seiring berkembangnya sistem pencatatan transaksi dan manajemen pelanggan. Data tersebut sangat potensial untuk dianalisis guna menghasilkan wawasan bisnis yang bernilai, seperti pengelompokan pelanggan berdasarkan perilaku atau preferensi tertentu. Salah satu metode yang efektif untuk melakukan pengelompokan ini adalah *Hierarchical Clustering Algorithm* (HCA), yaitu algoritma klusterisasi yang menyusun data ke dalam bentuk hierarki dengan struktur pohon, sehingga hubungan antar data dapat dipahami secara bertahap dan menyeluruh.

Namun, HCA memiliki kelemahan utama dalam hal efisiensi komputasi. Kompleksitas waktu HCA umumnya sebesar $O(n^2 \log n)$ atau lebih, tergantung pada metode dan implementasinya, di mana n adalah jumlah data. HCA memerlukan perhitungan jarak antar semua pasangan data dan proses penggabungan kluster yang dilakukan secara berulang. Hal ini menyebabkan waktu eksekusi meningkat drastis seiring bertambahnya jumlah data.

Untuk mengatasi keterbatasan tersebut, pemrosesan paralel diperlukan. Dengan pendekatan paralel, proses perhitungan jarak antar data dan penggabungan kluster dapat dibagi ke beberapa *rank* (proses) dan dijalankan secara bersamaan pada satu atau lebih *device* (komputer). Teknologi seperti MPI (*Message Passing Interface*) memungkinkan koordinasi antar proses pada lingkungan komputasi terdistribusi, sehingga waktu eksekusi dapat ditekan secara signifikan. Komputasi paralel tidak hanya meningkatkan kecepatan eksekusi, tetapi juga memungkinkan pemrosesan dataset yang jauh lebih besar daripada kapasitas satu komputer tunggal.

Penelitian ini secara khusus berfokus pada implementasi algoritma *Hierarchical Clustering* berbasis pemrosesan paralel menggunakan MPI, dengan tujuan untuk menguji dan menganalisis kinerja waktu eksekusi berdasarkan jumlah data yang diproses, jumlah *rank* atau proses paralel yang dijalankan, dan jumlah *device* atau komputer fisik yang digunakan. Dengan melakukan eksperimen pada berbagai kombinasi parameter tersebut, penelitian ini bertujuan untuk mengetahui sejauh mana pemrosesan paralel dapat meningkatkan efisiensi HCA serta mengidentifikasi konfigurasi yang optimal dalam konteks pengolahan data pelanggan mall berskala besar.

BAB II

DASAR TEORI

A. Pemrosesan MPI (*Message Passing Protocol*)

Message Passing Interface (MPI) adalah salah satu protokol komunikasi yang digunakan dalam pemrograman pada komputer paralel. Tujuan menggunakan MPI adalah memberikan kinerja yang tinggi, scalable dan mudah dimigrasikan dari platform komputasi yang satu ke platform komputasi yang lain. MPI merupakan protokol yang banyak digunakan dalam komputasi berkinerja tinggi (Wanasurya et al, 2017). Pemrosesan paralel menggunakan MPI memungkinkan pembagian data ke dalam beberapa proses yang bekerja secara bersamaan. MPI menyediakan berbagai fungsi komunikasi kolektif seperti MPI_Scatter, MPI_Gather, MPI_Bcast, dan MPI_Reduce untuk memfasilitasi pengiriman dan penerimaan data antar proses. Misalnya, MPI_Bcast digunakan untuk menyebarkan informasi dari satu proses ke seluruh proses lain dan umumnya dipakai untuk menginisialisasi struktur data aplikasi, sedangkan MPI_Reduce digunakan untuk menggabungkan nilai-nilai dari seluruh proses dan mengirimkan hasil akhirnya ke satu proses root. Dalam konteks arsitektur multi-core, implementasi fungsi-fungsi kolektif ini dapat dioptimalkan melalui penggunaan shared memory yang dapat meningkatkan kinerja hingga tiga kali lipat dibanding pendekatan tradisional berbasis point-to-point (Graham & Shipman, 2008).

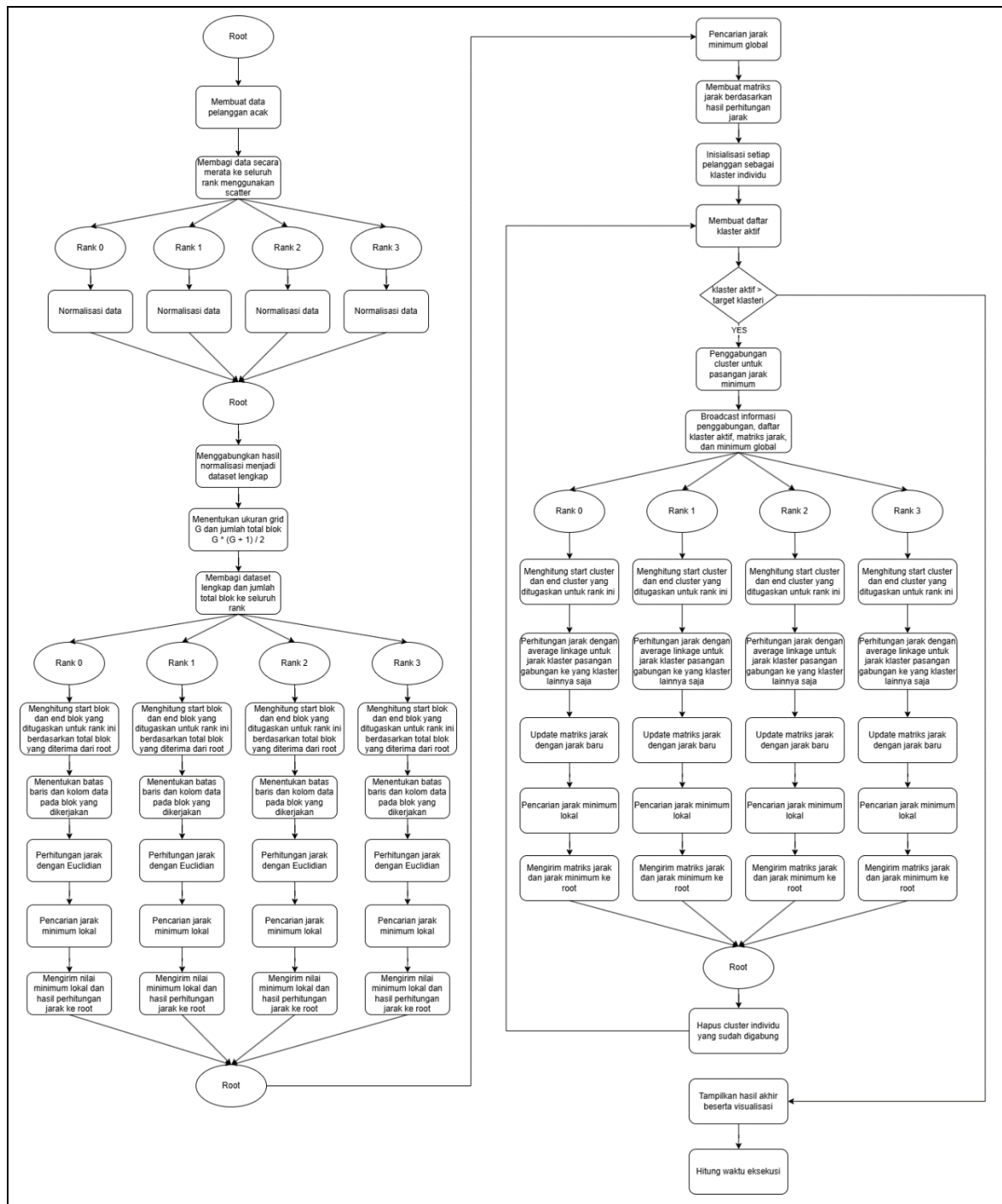
B. Heuristical *Clustering Algorithm* (HCA)

Hierarchical *clustering* merupakan salah satu pendekatan klasterisasi yang efektif digunakan untuk menemukan pengelompokan data yang optimal atau mendekati optimal, terutama pada data yang bersifat kompleks atau berukuran besar. Berbeda dengan algoritma klasterisasi konvensional seperti *K-Means* yang bersifat deterministik dan sensitif terhadap inisialisasi awal, hierarchical *clustering* membentuk klaster secara bertahap melalui pendekatan *bottom-up* (agglomerative) atau *top-down* (divisive) yang divisualisasikan dalam bentuk dendrogram. Pendekatan ini dinilai lebih fleksibel dalam menangani data dengan struktur atau pola klaster yang tidak beraturan. Salah satu algoritma yang umum digunakan dalam metode ini adalah *Agglomerative Nesting* (AGNES) yang menggabungkan objek-objek berdasarkan tingkat kemiripan dengan prinsip meminimalkan total variasi dalam klaster, serta *Divisive Analysis* (DIANA) yang membagi objek ke dalam klaster-klaster lebih kecil berdasarkan tingkat heterogenitasnya (Raihannabil, 2024).

BAB III

METODE

Program ini menggunakan “*Hierarchical Clustering Algorithm* dengan *Average Linkage*”. Dalam pemrosesan paralel, *hierarchical clustering* dapat diimplementasikan dengan membagi perhitungan jarak antar pasangan data ke beberapa prosesor, serta mempercepat proses penggabungan kluster secara bertahap menggunakan komunikasi antar node melalui MPI (*Message Passing Interface*) atau teknologi paralel lainnya.



Gambar 1. Flowchart Algoritma HCA secara Paralel

Berdasarkan *flowchart* di atas, algoritma dari metode tersebut sebagai berikut:

1. Inisialisasi dan Persiapan Data

- a. Semua Rank: Inisialisasi MPI, mengetahui rank dan size.
- b. Rank 0: Membuat data pelanggan acak sebanyak 1000 data atau sesuai dengan keperluan pengujian (usia (18–70), pendapatan tahunan (15k–137k), skor belanja (1–100). Data mentah (belum dinormalisasi) dibagi secara merata ke seluruh rank menggunakan Scatter.
- c. Semua Rank (Termasuk Rank 0): Menerima subset data mentah hasil scatter. Melakukan normalisasi lokal: usia $\rightarrow$ 3 kategori: remaja (<30), dewasa (30–59), lansia (60+), pendapatan $\rightarrow$ 2 kategori: rendah ($<70k$), tinggi ($\geq 70k$), pengeluaran $\rightarrow$ 2 kategori: rendah (<50), tinggi (≥ 50)
- d. Semua Rank: Mengirim hasil normalisasi lokal ke rank 0 menggunakan Gather.
- e. Rank 0: Menggabungkan hasil normalisasi dari semua rank menjadi dataset normalisasi lengkap. Menyebarkan kembali dataset yang sudah dinormalisasi lengkap ke seluruh rank menggunakan Broadcast.

2. Membuat Matriks Jarak Awal dengan Grid-Based Distribution

- a. Rank 0: Menentukan ukuran grid G berdasarkan akar kuadrat dari dua kali jumlah proses, kemudian membagi matriks jarak ke dalam blok-blok segitiga atas. Total blok yang dihasilkan adalah $G(G+1)/2$.
- b. Semua Rank: Setiap proses menerima sejumlah blok untuk diproses secara paralel. Di setiap blok yang ditugaskan, proses menghitung jarak Euclidean antar pasangan pelanggan dan mencari jarak minimum lokal beserta pasangan yang menghasilkan jarak tersebut.
- c. Semua Rank: Hasil jarak minimum lokal dan pasangan terkait dikumpulkan ke rank 0 menggunakan gather operation.
- d. Rank 0: Menentukan jarak minimum global dan pasangan global dari semua hasil lokal, kemudian membangun matriks jarak lengkap dari semua jarak yang telah dihitung oleh setiap proses.
- e. Rank 0: Matriks jarak lengkap beserta informasi minimum global disebar kembali ke semua proses menggunakan broadcast.

3. Inisialisasi Cluster Aktif

- a. Rank 0: Menginisialisasi setiap pelanggan sebagai cluster individual dengan ID unik dan ukuran 1. Membuat daftar cluster aktif yang berisi semua cluster individual.

- b. Rank 0: Daftar cluster aktif dibagikan ke semua proses menggunakan broadcast agar semua proses memiliki informasi cluster yang konsisten.
4. Iterasi Penggabungan Cluster dengan Average Linkage
 - a. Semua Rank: Memulai loop iterasi yang berlanjut hingga jumlah cluster mencapai target (5 cluster). Pada iterasi pertama, menggunakan pasangan minimum dari tahap pembangunan matriks awal.
 - b. Rank 0: Untuk setiap iterasi, menampilkan informasi cluster yang akan digabung, menyimpan riwayat penggabungan, dan melakukan penggabungan dengan menambahkan anggota cluster kedua ke cluster pertama serta memperbarui ukuran cluster.
5. Update Matriks Jarak dan Pencarian Jarak Minimum Paralel
 - a. Semua Rank: Setelah penggabungan cluster, pekerjaan untuk menghitung ulang jarak dari cluster yang baru terbentuk ke cluster lain dibagi secara paralel. Setiap proses menghitung jarak menggunakan Average Linkage untuk subset cluster yang ditugaskan.
 - b. Semua Rank: Hasil perhitungan jarak baru dikumpulkan ke rank 0 menggunakan gather operation untuk memperbarui matriks jarak.
 - c. Rank 0: Memperbarui matriks jarak dengan jarak-jarak baru yang telah dihitung, kemudian menyebarkan matriks yang telah diperbarui ke semua proses.
 - d. Semua Rank: Secara paralel melakukan pencarian pasangan cluster dengan jarak minimum untuk iterasi berikutnya. Setiap proses mengecek subset pasangan yang ditugaskan dan mencari minimum lokal.
 - e. Semua Rank: Hasil minimum lokal dikumpulkan ke rank 0 menggunakan gather untuk menentukan minimum global.
 - f. Rank 0: Menentukan jarak minimum global dan pasangan cluster untuk iterasi berikutnya, kemudian menyebarkan informasi ini ke semua proses.
6. Penghapusan Cluster dan Pengecekan Kondisi Berhenti
 - a. Semua Rank: Setelah penggabungan, cluster yang telah digabung (cluster kedua) dihapus dari daftar cluster aktif.
 - b. Semua Rank: Memeriksa apakah jumlah cluster aktif sudah mencapai target. Jika belum, iterasi dilanjutkan menggunakan hasil minimum yang telah dihitung sebelumnya tanpa perlu perhitungan ulang.
7. Visualisasi dan Analisis Hasil Akhir

- a. Rank 0: Setelah clustering selesai, menampilkan matriks jarak final dalam bentuk tabel dan heatmap. Menganalisis karakteristik setiap cluster berdasarkan rata-rata usia, pendapatan, dan pengeluaran anggotanya.
- b. Rank 0: Melakukan interpretasi bisnis untuk setiap cluster dan memberikan rekomendasi strategi pemasaran berdasarkan profil pelanggan di masing-masing cluster.
- c. Rank 0: Membuat visualisasi 3D scatter plot untuk menampilkan hasil clustering dalam ruang tiga dimensi (usia, pendapatan, pengeluaran) dan menyimpan hasil visualisasi.
- d. Rank 0: Menampilkan ringkasan waktu eksekusi untuk setiap tahap proses dan total waktu yang diperlukan untuk menyelesaikan seluruh proses clustering.

BAB IV

PENGUJIAN DAN PEMBAHASAN

A. Percobaan 1

- 1) Data Hasil Pengujian Waktu Eksekusi untuk Berbagai Jumlah Data terhadap Berbagai Jumlah Device Menggunakan 4 N

Banyak Device	100	200	300	400	500	600	700
1	43.9473	92.8141	230.0776	404.7956	666.6255	935.644	1010.274
2	25.4632	52.1302	116.6114	188.4286	350.8368	419.4625	636.257
3	24.1774	49.6699	110.9025	175.4986	346.846	411.3063	610.2354
4	18.3827	53.4351	106.3432	169.3746	305.6641	381.227	603.5332

Gambar 2. Data Pengujian 1 pada Percobaan 1

Gambar di atas berisi data waktu eksekusi untuk berbagai jumlah data (100 hingga 700) terhadap jumlah device yang digunakan (1 hingga 4 device). Baris pertama menunjukkan waktu eksekusi saat menggunakan 1 device, baris kedua untuk 2 device, baris ketiga untuk 3 device, dan baris keempat untuk 4 device. Setiap kolom mewakili ukuran data yang berbeda, yaitu 100, 200, 300, 400, 500, 600, dan 700. Nilai pada tabel merupakan hasil pengukuran waktu eksekusi untuk kombinasi jumlah device dan jumlah data tertentu.

- 2) Data Hasil Pengujian Waktu Eksekusi untuk Banyak Jumlah Device Menggunakan 4 N terhadap Berbagai Jumlah Data

Banyak Device	100	200	300	400	500	600	700
1	43.9473	92.8141	230.0776	404.7956	666.6255	935.644	1010.274
2	25.4632	52.1302	116.6114	188.4286	350.8368	419.4625	636.257
3	24.1774	49.6699	110.9025	175.4986	346.846	411.3063	610.2354
4	18.3827	53.4351	106.3432	169.3746	305.6641	381.227	603.5332

Gambar 3. Data Pengujian 2 pada Percobaan 1

Gambar di atas berisi data waktu eksekusi untuk berbagai jumlah data (100 hingga 700) terhadap jumlah device yang digunakan (1 hingga 4 device). Baris pertama menunjukkan waktu eksekusi saat menggunakan 1 device, baris kedua untuk 2 device, baris ketiga untuk 3 device, dan baris keempat untuk 4 device. Setiap kolom mewakili ukuran data yang berbeda, yaitu 100, 200, 300, 400, 500, 600, dan 700. Nilai pada tabel merupakan hasil pengukuran waktu eksekusi untuk kombinasi jumlah device dan jumlah data tertentu.

- 3) Data Hasil Pengujian Waktu Eksekusi untuk Berbagai Jumlah N terhadap 500 Data

Jumlah Data	4	8	12	16	20
500	666.626	334.172	274.982	254.662	224.276

Gambar 4. Data Pengujian 3 pada Percobaan 1

Gambar di atas menyajikan data waktu eksekusi untuk jumlah data tetap sebesar 500, dengan variasi jumlah N (jumlah node atau proses) yaitu 4, 8, 12, 16, dan 20. Baris pertama menunjukkan nilai waktu eksekusi yang dicapai untuk masing-masing jumlah N. Setiap nilai menunjukkan lama waktu yang dibutuhkan dalam satuan tertentu (misalnya milidetik atau detik) untuk menyelesaikan proses dengan konfigurasi jumlah N yang berbeda.

B. Percobaan 2

- 1) Data Hasil Pengujian Waktu Eksekusi untuk Berbagai Jumlah Data terhadap Berbagai Jumlah Device Menggunakan 4 N

Banyak device	100	200	300	400	500	600	700
1	43.3982	92.6987	230.176	404.539	667.257	936.005	1011.36
2	24.9045	52.1267	116.459	187.89	351.498	418.832	636.94
3	23.987	49.4765	110.091	175.476	346.729	410.761	609.36
4	18.0938	45.3487	105.903	170.076	305.988	380.344	603.285

Gambar 5. Data Pengujian 1 pada Percobaan 2

Gambar di atas menampilkan hasil pengujian waktu eksekusi untuk jumlah data yang bervariasi, yaitu dari 100 hingga 700, terhadap jumlah device yang digunakan (1 hingga 4 device), dengan jumlah N tetap sebanyak 4. Setiap baris merepresentasikan waktu eksekusi untuk konfigurasi jumlah device tertentu, sedangkan setiap kolom menunjukkan ukuran data yang diuji. Nilai-nilai pada tabel menunjukkan lama waktu eksekusi yang dicapai pada masing-masing kombinasi jumlah data dan jumlah device.

- 2) Pengujian Waktu Eksekusi untuk Banyak Jumlah Device Menggunakan 4 N terhadap Berbagai Jumlah Data

Banyak device	100	200	300	400	500	600	700
1	43.3982	92.6987	230.176	404.539	667.257	936.005	1011.36
2	24.9045	52.1267	116.459	187.89	351.498	418.832	636.94
3	23.987	49.4765	110.091	175.476	346.729	410.761	609.36
4	18.0938	45.3487	105.903	170.076	305.988	380.344	603.285

Gambar 6. Data Pengujian 2 pada Percobaan 2

Gambar di atas menyajikan data hasil pengujian waktu eksekusi untuk berbagai jumlah device (1 hingga 4 device) dengan jumlah N tetap sebanyak 4, terhadap jumlah data yang bervariasi dari 100 hingga 700. Setiap baris pada tabel merepresentasikan jumlah device tertentu, sedangkan setiap kolom menunjukkan jumlah data yang digunakan dalam pengujian. Nilai-nilai pada tabel menunjukkan lama waktu eksekusi yang dibutuhkan untuk setiap kombinasi jumlah device dan jumlah data tertentu.

3) Pengujian Waktu Eksekusi untuk Berbagai Jumlah N terhadap 500 Data

Jumlah Data	4	8	12	16	20
500	666.257	332.843	274.562	251.522	224.762

Gambar 7. Data Pengujian 3 pada Percobaan 1

Gambar di atas menampilkan hasil pengujian waktu eksekusi untuk jumlah data tetap sebesar 500 terhadap variasi jumlah N, yaitu 4, 8, 12, 16, dan 20. Baris pertama menunjukkan jumlah N yang digunakan, sementara baris kedua menunjukkan waktu eksekusi yang diperoleh untuk masing-masing jumlah N tersebut. Nilai-nilai dalam tabel merepresentasikan lama waktu yang dibutuhkan untuk menyelesaikan proses pada setiap konfigurasi jumlah N tertentu.

C. Percobaan 3

1) Data Hasil Pengujian Waktu Eksekusi untuk Berbagai Jumlah Data terhadap Berbagai Jumlah Device Menggunakan 4 N

Banyak device	100	200	300	400	500	600	700
1	43.983	94.721	230.948	403.812	667.001	935.534	1010.76
2	25.581	52.904	115.883	188.442	350.802	418.192	636.478
3	22.3804	48.5675	110.562	175.024	346.157	410.273	609.011
4	17.587	46.2437	105.293	170.642	306.458	380.732	602.798

Gambar 8. Data Pengujian 1 pada Percobaan 3

Gambar di atas menyajikan data hasil pengujian waktu eksekusi untuk berbagai jumlah data, yaitu mulai dari 100 hingga 700, terhadap jumlah device yang digunakan sebanyak 1 hingga 4, dengan jumlah N tetap sebanyak 4. Baris-baris pada tabel mewakili jumlah device yang berbeda, sedangkan kolom-kolom mewakili ukuran data yang diuji. Nilai-nilai di dalam tabel menunjukkan waktu eksekusi yang diperlukan pada setiap kombinasi jumlah device dan jumlah data yang ditentukan.

2) Data Hasil Pengujian Waktu Eksekusi untuk Banyak Jumlah Device Menggunakan 4 N terhadap Berbagai Jumlah Data

Banyak device	100	200	300	400	500	600	700
1	43.983	94.721	230.948	403.812	667.001	935.534	1010.76
2	25.581	52.904	115.883	188.442	350.802	418.192	636.478
3	22.3804	48.5675	110.562	175.024	346.157	410.273	609.011
4	17.587	46.2437	105.293	170.642	306.458	380.732	602.798

Gambar 9. Data Pengujian 2 pada Percobaan 3

Gambar di atas menunjukkan hasil pengujian waktu eksekusi untuk berbagai jumlah device (1 hingga 4) terhadap jumlah data yang bervariasi mulai dari 100 hingga 700, dengan jumlah N tetap sebanyak 4. Setiap baris mewakili jumlah

device yang digunakan dalam pengujian, sedangkan setiap kolom menunjukkan ukuran data yang diuji. Nilai pada tabel merepresentasikan lama waktu eksekusi yang dicatat untuk setiap kombinasi jumlah device dan jumlah data tertentu.

3) Data Hasil Pengujian Waktu Eksekusi untuk Berbagai Jumlah N terhadap 500 Data

Jumlah data	4	8	12	16	20
500	667.923	333.016	275.381	250.76	225.912

Gambar 10. Data Pengujian 3 pada Percobaan 3

Gambar di atas menampilkan hasil pengujian waktu eksekusi untuk jumlah data tetap sebesar 500 terhadap variasi jumlah N, yaitu 4, 8, 12, 16, dan 20. Baris pertama menunjukkan nilai jumlah N yang digunakan dalam pengujian, sedangkan baris kedua berisi waktu eksekusi yang tercatat untuk masing-masing nilai N tersebut. Setiap nilai pada tabel merepresentasikan lama waktu yang dibutuhkan untuk menyelesaikan proses komputasi pada konfigurasi jumlah N tertentu dengan data berjumlah 500.

D. Rata-rata Percobaan 1, 2 dan 3

1) Pengujian Waktu Eksekusi untuk Berbagai Jumlah Data terhadap Berbagai Jumlah Device Menggunakan 4 N

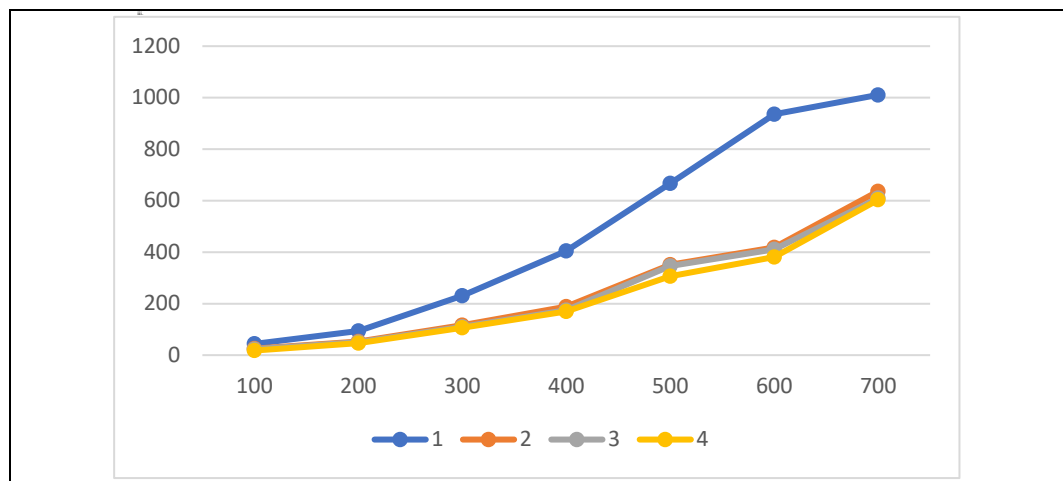
i. Rata-Rata Percobaan 1, 2 dan 3

Banyak Device	100	200	300	400	500	600	700
1	43.77617	93.41127	230.4004	404.3822	666.9612	935.7276	1010.798
2	25.31623	52.82193	116.3178	188.2536	351.0456	418.8288	636.5584
3	23.51493	50.05807	110.5186	175.333	346.5774	410.7801	609.5355
4	18.02117	47.08743	105.8465	170.031	306.0367	380.7677	603.2054

Gambar 11. Rata-Rata Data Pengujian 1 pada Percobaan 1, 2 dan 3

Gambar di atas menampilkan rata-rata waktu eksekusi dari hasil pengujian untuk berbagai jumlah data (100 hingga 700) terhadap jumlah device (1 hingga 3 device) menggunakan 4 node (N). Setiap baris merepresentasikan percobaan ke-1 hingga ke-3, dan setiap kolom menunjukkan waktu rata-rata eksekusi untuk jumlah data tertentu. Nilai-nilai dalam tabel mencerminkan waktu yang dibutuhkan oleh sistem untuk menyelesaikan proses eksekusi data dengan konfigurasi jumlah device tertentu pada masing-masing percobaan, yang diukur secara terpisah untuk tiap skenario beban data.

ii. Grafik



Gambar 12. Grafik Rata-Rata Data Pengujian 1 pada Percobaan 1, 2 dan 3

Berdasarkan Grafik Perbandingan Waktu Eksekusi untuk Berbagai Jumlah Data terhadap Berbagai Jumlah Device menggunakan 4 N, terlihat bahwa waktu eksekusi meningkat seiring bertambahnya jumlah data. Namun, penggunaan lebih banyak perangkat (device) secara konsisten menghasilkan waktu eksekusi yang lebih rendah dibandingkan penggunaan 1 perangkat. Selisih waktu paling signifikan terlihat pada jumlah data besar (500 hingga 700), menunjukkan bahwa paralelisasi dengan lebih banyak perangkat menjadi semakin efektif saat menangani data berukuran besar. Sementara itu, untuk data kecil, perbedaan waktu antar jumlah perangkat tidak terlalu signifikan. Hal ini menunjukkan bahwa manfaat penambahan perangkat baru terasa optimal pada skenario komputasi dengan volume data besar.

2) Pengujian Waktu Eksekusi untuk Banyak Jumlah Device Menggunakan 4 N terhadap Berbagai Jumlah Data

i. Rata-Rata Percobaan 1, 2 dan 3

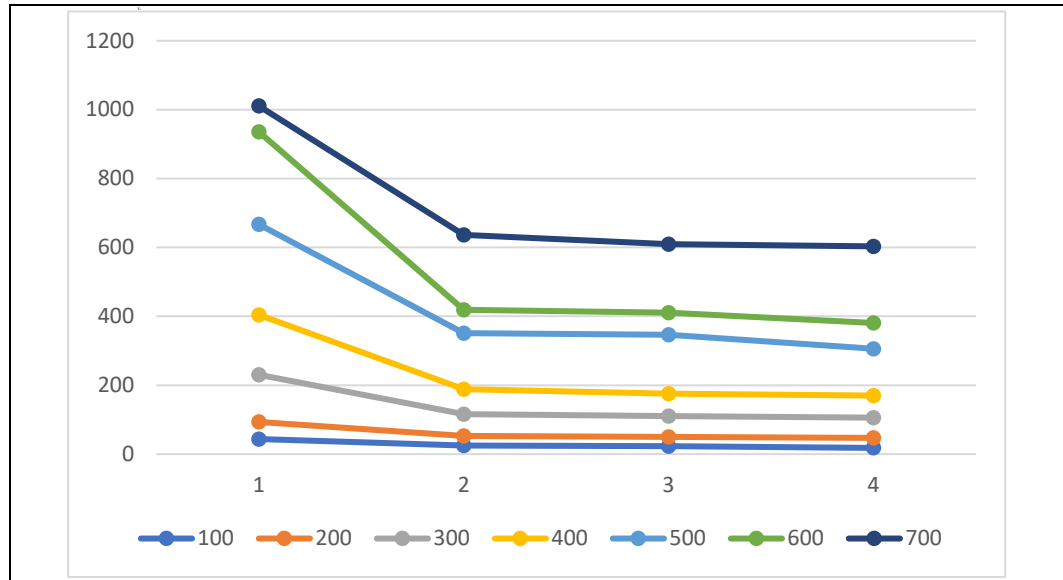
Banyak Device	100	200	300	400	500	600	700
1	43.77617	93.41127	230.4004	404.3822	666.9612	935.7276	1010.798
2	25.31623	52.82193	116.3178	188.2536	351.0456	418.8288	636.5584
3	23.51493	50.05807	110.5186	175.333	346.5774	410.7801	609.5355
4	18.02117	47.08743	105.8465	170.031	306.0367	380.7677	603.2054

Gambar 13. Rata-Rata Data Pengujian 2 pada Percobaan 1, 2 dan 3

Gambar di atas menyajikan data rata-rata waktu eksekusi dari hasil pengujian terhadap berbagai jumlah device (1 hingga 3 perangkat) menggunakan konfigurasi 4 node (N), dengan pengujian dilakukan pada berbagai jumlah data (dari 100 hingga 700). Setiap kolom mewakili jumlah device, sementara setiap baris menunjukkan waktu rata-rata eksekusi yang dicatat untuk masing-masing jumlah data pada

percobaan ke-1, ke-2, dan ke-3. Nilai-nilai dalam tabel menunjukkan rata-rata durasi proses yang dibutuhkan untuk menyelesaikan tugas komputasi berdasarkan kombinasi jumlah device dan data yang digunakan.

ii. Grafik



Gambar 14. Grafik Rata-Rata Data Pengujian 2 pada Percobaan 1, 2 dan 3

Berdasarkan Grafik Perbandingan Waktu Eksekusi untuk Banyak Jumlah Device menggunakan 4 N terhadap Berbagai Jumlah Data di atas, terlihat bahwa peningkatan jumlah perangkat secara konsisten menurunkan waktu eksekusi, terutama pada jumlah data besar. Penurunan waktu paling signifikan terjadi saat beralih dari 1 ke 2 perangkat, khususnya pada data berukuran 500, 600, dan 700, yang menunjukkan efektivitas distribusi beban komputasi.

Salah satu faktor pendukung efisiensi ini adalah optimalisasi penggunaan shared memory antar perangkat. Dengan shared memory, data dapat diakses bersama secara cepat oleh seluruh perangkat yang terlibat, sehingga proses pertukaran informasi dan sinkronisasi menjadi lebih efisien. Hal ini memungkinkan perangkat bekerja secara paralel tanpa perlu memuat ulang data yang sama, sehingga proses eksekusi dapat dipercepat secara signifikan, terutama saat menangani data dalam jumlah besar.

3) Pengujian Waktu Eksekusi untuk Berbagai Jumlah N terhadap 500 Data

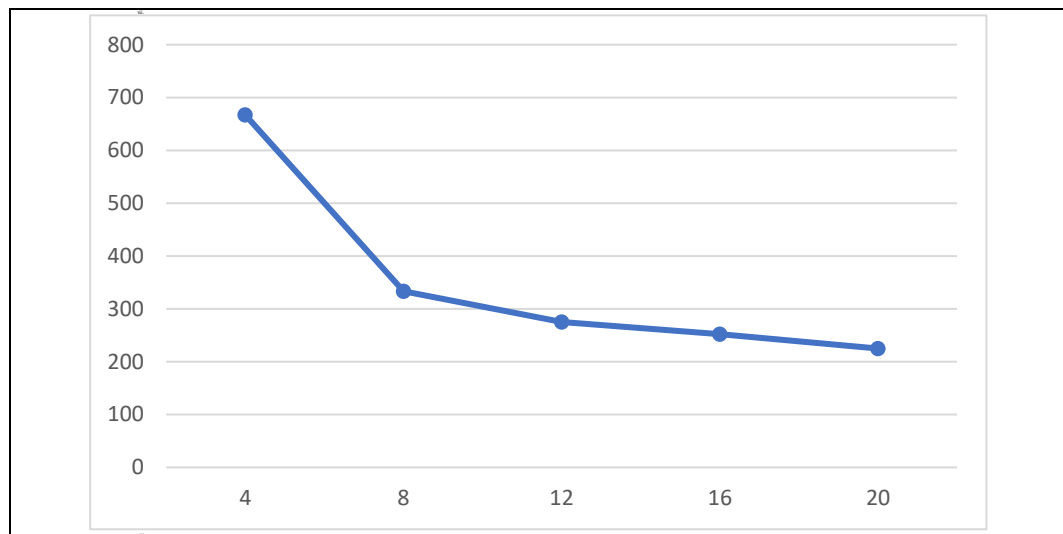
i. Rata-Rata Percobaan 1, 2 dan 3

Jumlah data	4	8	12	16	20
500	666.935	333.344	274.975	252.315	224.983

Gambar 15. Rata-Rata Data Pengujian 3 pada Percobaan 1, 2 dan 3

Gambar di atas menyajikan data rata-rata waktu eksekusi untuk jumlah data tetap sebesar 500, terhadap variasi jumlah N (node) yaitu 4, 8, 12, 16, dan 20, berdasarkan hasil percobaan ke-1, ke-2, dan ke-3. Baris pertama menunjukkan jumlah data yang digunakan dalam pengujian, sedangkan baris kedua memuat nilai waktu rata-rata eksekusi yang dibutuhkan untuk setiap konfigurasi jumlah node yang berbeda. Setiap nilai mencerminkan durasi waktu yang diperlukan untuk menyelesaikan proses pada 500 data dengan jumlah node tertentu.

ii. Grafik



Gambar 15. Grafik Rata-Rata Data Pengujian 3 pada Percobaan 1, 2 dan 3

Berdasarkan Grafik Perbandingan Waktu Eksekusi untuk Berbagai Jumlah N terhadap 500 Data di atas, terlihat bahwa peningkatan jumlah N (misalnya jumlah thread atau core) secara signifikan menurunkan waktu eksekusi. Penurunan paling tajam terjadi saat N meningkat dari 4 ke 8, menunjukkan efisiensi paralelisasi yang optimal pada tahap tersebut. Setelah N lebih dari 8, penurunan waktu eksekusi masih terjadi, namun cenderung melambat, mengindikasikan adanya titik batas efisiensi di mana penambahan N tidak lagi memberikan peningkatan performa yang signifikan. Hal ini dapat disebabkan oleh overhead komunikasi dan koordinasi antar thread yang semakin besar seiring jumlah N yang meningkat.

BAB V

KESIMPULAN

Berdasarkan hasil percobaan, dapat disimpulkan bahwa penggunaan pendekatan MPI (Message Passing Interface) dan komputasi paralel secara nyata meningkatkan efisiensi proses komputasi, terutama pada pemrosesan data berukuran besar. MPI memungkinkan pembagian beban kerja ke beberapa perangkat atau prosesor secara terdistribusi, sehingga waktu eksekusi dapat ditekan secara signifikan dibandingkan dengan pemrosesan secara tunggal. Namun, meskipun penambahan perangkat dapat mempercepat proses, terdapat batas efisiensi di mana peningkatan jumlah perangkat tidak lagi memberikan pengurangan waktu yang sebanding, karena adanya overhead komunikasi dan koordinasi antar-proses. Oleh karena itu, pemanfaatan MPI dan sistem paralel sangat efektif untuk mempercepat proses komputasi skala besar, namun tetap memerlukan pertimbangan optimal dalam jumlah perangkat yang digunakan.

Lebih lanjut, berdasarkan grafik perbandingan waktu eksekusi terhadap berbagai jumlah N (jumlah thread atau core) untuk data sebanyak 500, terlihat bahwa peningkatan jumlah N secara signifikan menurunkan waktu eksekusi, terutama saat meningkat dari 4 ke 8. Penurunan ini mencerminkan efisiensi paralelisasi yang sangat optimal pada tahap awal. Namun, setelah N melebihi 8, penurunan waktu tetap terjadi tetapi melambat, menandakan bahwa titik batas efisiensi telah tercapai dan penambahan N selanjutnya tidak memberikan dampak signifikan karena beban koordinasi dan komunikasi yang meningkat antar thread.

Selain itu, grafik perbandingan waktu eksekusi terhadap banyaknya perangkat menggunakan $4N$ untuk berbagai jumlah data juga menunjukkan bahwa peningkatan jumlah perangkat secara konsisten menurunkan waktu eksekusi, terutama pada data berukuran besar seperti 500 hingga 700. Penurunan waktu paling mencolok terjadi saat transisi dari 1 ke 2 perangkat, yang menunjukkan efektivitas distribusi beban komputasi. Efisiensi ini didukung oleh pemanfaatan shared memory, di mana data dapat diakses bersama oleh perangkat secara cepat, mempercepat proses sinkronisasi dan eksekusi paralel tanpa perlu memuat ulang data secara terpisah.

Akhirnya, dari grafik perbandingan waktu eksekusi untuk berbagai jumlah data terhadap berbagai jumlah perangkat (menggunakan $4N$), dapat dilihat bahwa waktu eksekusi meningkat seiring bertambahnya jumlah data. Namun, semakin banyak perangkat yang digunakan, waktu eksekusi cenderung lebih rendah, terutama untuk data dalam jumlah besar. Hal ini menunjukkan bahwa paralelisasi menjadi semakin efektif saat menangani beban

komputasi besar, sementara untuk data kecil, efisiensi penambahan perangkat tidak terlalu terasa. Keseluruhan hasil ini menegaskan bahwa kombinasi antara jumlah N dan perangkat yang tepat sangat penting untuk mencapai kinerja optimal dalam komputasi paralel berbasis MPI.

REFERENSI

- Wanasurya, A. L., Kartawidjaja, M. A., & Mulyanti, S. (2017). Komputasi paralel untuk pengolahan prestasi akademik mahasiswa. *Jurnal Teknologi Elektro*, 8(3), 170–175.
- Graham, R. L., & Shipman, G. (2008). MPI support for multi-core architectures: Optimized shared memory collectives. In A. Lastovetsky, T. Kechadi, & J. Dongarra (Eds.), *Recent Advances in Parallel Virtual Machine and Message Passing Interface. EuroPVM/MPI 2008. Lecture Notes in Computer Science* (Vol. 5205, pp. 130–140). Springer.
- Raihannabil, S. D. (2024). Penerapan metode hierarchical *clustering* untuk klasterisasi provinsi di Indonesia berdasarkan indikator status gizi anak baduta (bawah dua tahun) tahun 2023. *Emerging Statistics and Data Science Journal*, 2(3), 424–434.

LAMPIRAN

A. ANALISIS CODE

```
from mpi4py import MPI
import numpy as np
import pandas as pd
import math
import sys
import time
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns
```

Script tersebut melakukan *import library* yang diperlukan untuk menjalankan program komputasi numerik dan visualisasi data dalam konteks pemrosesan paralel. *Library* “mpi4py” digunakan untuk komunikasi antar proses dalam komputasi paralel menggunakan protokol Message Passing Interface (MPI), memungkinkan pembagian tugas secara efisien pada beberapa prosesor. “numpy” digunakan untuk melakukan operasi numerik dan manipulasi array secara efisien. pandas diimpor untuk mempermudah pengelolaan dan analisis data dalam bentuk tabel. “math” digunakan untuk operasi matematika dasar, sementara “sys” digunakan untuk menangani argumen sistem atau pengaturan lain yang berkaitan dengan sistem operasi. “time” dipakai untuk mencatat waktu eksekusi program, berguna dalam analisis performa. Untuk keperluan visualisasi, digunakan “matplotlib.pyplot” dan “seaborn”. “matplotlib.colors.ListedColormap” memungkinkan pembuatan palet warna kustom untuk visualisasi data, dan “mpl_toolkits.mplot3d.Axes3D” digunakan untuk membuat plot dalam tiga dimensi. “seaborn” melengkapi visualisasi dengan tampilan grafik yang lebih menarik dan informatif.

```
class Customer:
    def __init__(self, id, umur, pendapatan, pengeluaran):
        self.id = id
        self.umur = umur
        self.pendapatan = pendapatan
        self.pengeluaran = pengeluaran
        self.fitur = [umur, pengeluaran, pendapatan]
        self.cluster_id = id
```

Script tersebut mendefinisikan sebuah *class* bernama “Customer” yang digunakan untuk merepresentasikan data individu pelanggan dalam konteks analisis data atau pengelompokan. “def __init__” menerima empat parameter utama yaitu “id”, “umur”, “pendapatan”, dan “pengeluaran”, yang kemudian disimpan sebagai atribut objek. Atribut fitur dibuat sebagai sebuah list yang berisi tiga elemen numerik: “umur”, “pengeluaran”, dan “pendapatan”, urutan ini kemungkinan dipilih untuk digunakan dalam proses pengolahan data seperti klasifikasi atau *clustering*, di mana fitur harus berupa vektor numerik. Atribut “cluster_id” diinisialisasi dengan id pelanggan, kemungkinan untuk digunakan dalam proses *clustering* sebagai identitas awal sebelum dikelompokkan ke dalam cluster tertentu.

```
class Cluster:
    def __init__(self, id, anggota=None):
        self.id = id
        self.anggota = anggota if anggota else []
        self.size = len(self.anggota)
```

Script tersebut mendefinisikan sebuah *class* bernama “Cluster” yang berfungsi untuk merepresentasikan satu cluster dalam konteks algoritma pengelompokan data, seperti *clustering*. “Def __init__” menerima dua parameter yaitu “id” sebagai identitas unik dari cluster, dan “anggota” yang merupakan daftar objek. Jika tidak ada data anggota yang diberikan saat inisialisasi, maka atribut anggota akan otomatis diisi dengan list kosong. Selanjutnya, atribut “size” menyimpan jumlah anggota dalam cluster, yang dihitung berdasarkan panjang list anggota. *Class* ini dirancang untuk menyimpan dan mengelola informasi dasar tentang suatu cluster, seperti identitas dan jumlah anggotanya, sehingga dapat digunakan dalam proses analisis atau evaluasi hasil *clustering*.

```
class HasilClustering:
    def __init__(self):
        self.merge_history = []
        self.jarak = []
        self.clusters_at_each_step = []
```

Script tersebut mendefinisikan sebuah *class* bernama “HasilClustering” yang digunakan untuk menyimpan dan melacak proses serta hasil dari algoritma hierarchical *clustering* atau metode pengelompokan bertahap lainnya. “Def __init__” menginisialisasi tiga atribut utama berupa list kosong: “merge_history”, “jarak”, dan “clusters_at_each_step”. Atribut “merge_history” digunakan untuk mencatat history

penggabungan antar cluster pada setiap langkah proses *clustering*, biasanya berupa pasangan cluster yang digabung. Atribut “jarak” menyimpan nilai jarak atau kemiripan antar cluster yang digunakan sebagai dasar penggabungan pada setiap tahap. Sedangkan “clusters_at_each_step” menyimpan kondisi atau komposisi cluster setelah setiap langkah penggabungan, memungkinkan analisis bagaimana struktur data berkembang dari banyak cluster kecil hingga menjadi satu atau beberapa cluster besar.

```
def normalisasi_data(customers):  
    for customer in customers:  
        if 18 <= customer.umur <= 25:  
            customer.fitur[0] = 1  
        elif 26 <= customer.umur <= 49:  
            customer.fitur[0] = 2  
        elif 50 <= customer.umur <= 70:  
            customer.fitur[0] = 3  
  
        if customer.pendapatan <= 7600000:  
            customer.fitur[1] = 1  
        else:  
            customer.fitur[1] = 2  
  
        if customer.pengeluaran <= 50:  
            customer.fitur[2] = 1  
        else:  
            customer.fitur[2] = 2
```

Script tersebut mendefinisikan sebuah fungsi bernama “normalisasi_data” yang bertujuan untuk melakukan normalisasi atau transformasi data pada objek-objek “Customer” dalam sebuah daftar. Fungsi ini mengklasifikasikan nilai atribut “umur”, “pengeluaran”, dan “pendapatan” ke dalam kategori numerik yang disederhanakan, agar lebih mudah diolah oleh algoritma analisis data seperti *clustering*. Umur dibagi menjadi tiga kelompok umur: 18–25 dikodekan menjadi 1, 26–49 menjadi 2, dan 50–70 menjadi 3. Pendapatan dibagi menjadi dua *class*: $\leq 7.600.000$ diberi nilai 1, dan sisanya 2. Pengeluaran pun diklasifikasikan menjadi dua kategori: ≤ 50 menjadi 1, selebihnya 2. Nilai-nilai hasil normalisasi ini kemudian disimpan langsung ke dalam atribut fitur milik setiap objek “Customer”, yang merepresentasikan vektor fitur untuk keperluan analisis lanjutan.

```
def euclidean(customer1, customer2):
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(customer1.fitur,
customer2.fitur)))
```

Script tersebut mendefinisikan sebuah fungsi bernama “Euclidean” yang digunakan untuk menghitung jarak Euclidean antara dua objek “Customer” berdasarkan vektor fitur mereka. Fungsi ini menerima dua parameter, “customer1” dan “customer2” yang diasumsikan memiliki atribut fitur berupa list angka. Dengan menggunakan fungsi “zip”, pasangan elemen dari kedua vektor fitur dibandingkan satu per satu, selisih tiap elemen dikuadratkan, lalu dijumlahkan, dan hasilnya diakar kuadrat menggunakan “math.sqrt”. Hasil akhir berupa nilai jarak Euclidean, yaitu ukuran seberapa jauh dua pelanggan berada satu sama lain dalam ruang fitur.

```
def average_linkage(cluster1, cluster2, customers):
    total_jarak = 0.0
    total_pasangan = 0

    for member1_id in cluster1.anggota:
        for member2_id in cluster2.anggota:
            customer1 = customers[member1_id]
            customer2 = customers[member2_id]
            jarak = euclidean(customer1, customer2)
            total_jarak += jarak
            total_pasangan += 1

    if total_pasangan > 0:
        return total_jarak / total_pasangan
    else:
        return float('inf')
```

Script tersebut mendefinisikan sebuah fungsi bernama “average_linkage” yang digunakan untuk menghitung jarak antar dua cluster menggunakan metode *average linkage*, yaitu rata-rata jarak Euclidean antar seluruh pasangan anggota dari dua cluster. Fungsi ini menerima tiga parameter: “cluster1” dan “cluster2” yang merupakan objek cluster berisi daftar ID anggota, serta customers yang merupakan list atau dictionary objek “Customer”. Fungsi ini akan menelusuri semua kombinasi pasangan anggota dari cluster1 dan cluster2, lalu mengambil objek Customer berdasarkan ID masing-masing. Untuk setiap pasangan, jarak Euclidean dihitung menggunakan fungsi euclidean, lalu dijumlahkan ke dalam total_jarak, sambil menghitung total pasangan yang diproses. Setelah semua pasangan dihitung, hasil akhirnya adalah nilai rata-rata jarak tersebut, kecuali jika tidak ada pasangan yang valid, dalam hal ini fungsi mengembalikan tak hingga “(float('inf'))” untuk menandai ketidakberartian pengukuran.

```

def buat_matriks_jarak_awal_blok(customers, comm, rank, size):
    n = len(customers)
    matriks_jarak = np.full((n, n), float('inf'))

    G = int(math.ceil(math.sqrt(size * 2)))
    if G < 2:
        G = 2

    total_blok = G * (G + 1) // 2

    if rank == 0:
        print(f"Menggunakan grid {G}x{G} dengan {total_blok} blok untuk {size} proses")

    blok_per_rank = total_blok // size
    sisa_blok = total_blok % size

    if rank < sisa_blok:
        blok_untuk_rank_ini = blok_per_rank + 1
        start_blok = rank * (blok_per_rank + 1)
    else:
        blok_untuk_rank_ini = blok_per_rank
        start_blok = sisa_blok * (blok_per_rank + 1) + (rank - sisa_blok)
    * blok_per_rank

    end_blok = start_blok + blok_untuk_rank_ini

    semua_blok = []
    for baris_blok in range(G):
        for col_blok in range(baris_blok, G):
            semua_blok.append((baris_blok, col_blok))

    blok_rank_ini = semua_blok[start_blok:end_blok]

    local_distances = []
    local_min_distance = float('inf')
    local_min_pair = (-1, -1)

    for baris_blok, col_blok in blok_rank_ini:
        baris_start = baris_blok * n // G
        baris_end = min((baris_blok + 1) * n // G, n) - 1
        kolom_start = col_blok * n // G
        kolom_end = min((col_blok + 1) * n // G, n) - 1

        for i in range(baris_start, baris_end + 1):
            j_start = max(i + 1, kolom_start)
            for j in range(j_start, kolom_end + 1):
                if i < j:
                    dist = euclidean(customers[i], customers[j])
                    local_distances.append((i, j, dist))

                    if dist < local_min_distance:
                        local_min_distance = dist
                        local_min_pair = (i, j)

        print(f"Rank {rank}: Memproses {len(blok_rank_ini)} blok, Jarak minimum lokal: {local_min_distance:.4f}")

    all_min_results = comm.gather((local_min_distance, local_min_pair), root=0)

```

```

global_min_distance = float('inf')
global_min_pair = (-1, -1)

if rank == 0:
    for dist, pair in all_min_results:
        if dist < global_min_distance:
            global_min_distance = dist
            global_min_pair = pair

    print(f"Jarak minimum global: {global_min_distance:.4f}")
    print(f"Pasangan minimum global: Customer {global_min_pair[0]+1} dan
Customer {global_min_pair[1]+1}")

    global_result = comm.bcast((global_min_distance, global_min_pair),
root=0)
    global_min_distance, global_min_pair = global_result

    all_distances = comm.gather(local_distances, root=0)

if rank == 0:
    total_pairs_calculated = 0
    for proc_distances in all_distances:
        for i, j, dist in proc_distances:
            matriks_jarak[i][j] = dist
            matriks_jarak[j][i] = dist
            total_pairs_calculated += 1

    np.fill_diagonal(matriks_jarak, 0.0)

    expected_pairs = n * (n - 1) // 2
    print(f"Total pasangan dihitung: {total_pairs_calculated}")

    if total_pairs_calculated != expected_pairs:
        print(f"WARNING: Tidak semua pasangan telah dihitung!")

    matriks_jarak = comm.bcast(matriks_jarak, root=0)

    return matriks_jarak, global_min_distance, global_min_pair

```

Script tersebut menghitung matriks jarak antar pelanggan secara paralel menggunakan pendekatan pembagian blok segitiga atas pada matriks. *Script* tersebut membagi matriks jarak ke dalam beberapa blok berdasarkan jumlah proses, lalu mendistribusikan blok-blok tersebut secara merata ke setiap proses (rank). Masing-masing proses menghitung jarak Euclidean antar pelanggan dalam blok yang menjadi bagiannya, kemudian menentukan jarak minimum lokal dan mengirimkannya ke proses utama (rank 0). *Script* tersebut kemudian menggabungkan hasil dari semua proses untuk menemukan jarak minimum global dan pasangan pelanggan yang memiliki jarak terdekat. Setelah itu, seluruh jarak yang telah dihitung dikumpulkan untuk membentuk matriks jarak penuh, yang kemudian dibagikan ke seluruh proses agar semua memiliki salinan yang sama.

```

def update_distance_matrix_and_find_minimum(matriks_jarak,
merged_cluster_idx, removed_cluster_idx,
cluster_aktif, customers, comm,
rank, size):
    n = len(cluster_aktif)

    clusters_per_proc = n // size
    remaining_clusters = n % size

    if rank < remaining_clusters:
        clusters_for_this_proc = clusters_per_proc + 1
        start_cluster = rank * (clusters_per_proc + 1)
    else:
        clusters_for_this_proc = clusters_per_proc
        start_cluster = remaining_clusters * (clusters_per_proc + 1) +
(rank - remaining_clusters) * clusters_per_proc

    end_cluster = start_cluster + clusters_for_this_proc

    local_distances_update = []
    merged_cluster = cluster_aktif[merged_cluster_idx]

    for i in range(start_cluster, min(end_cluster, n)):
        if i != merged_cluster_idx and i != removed_cluster_idx:
            other_cluster = cluster_aktif[i]
            dist = average_linkage(merged_cluster, other_cluster,
customers)
            local_distances_update.append((i, dist))

    all_update_distances = comm.gather(local_distances_update, root=0)

    if rank == 0:
        for proc_distances in all_update_distances:
            for cluster_idx, dist in proc_distances:
                actual_merged_idx = merged_cluster.id
                actual_other_idx = cluster_aktif[cluster_idx].id
                matriks_jarak[actual_merged_idx][actual_other_idx] = dist
                matriks_jarak[actual_other_idx][actual_merged_idx] = dist

                actual_merged_idx = merged_cluster.id
                matriks_jarak[actual_merged_idx][actual_merged_idx] = 0.0

    matriks_jarak = comm.bcast(matriks_jarak, root=0)

    n_after_removal = n - 1
    total_pairs = n_after_removal * (n_after_removal - 1) // 2

    if total_pairs == 0:
        return matriks_jarak, float('inf'), (-1, -1)

    pairs_per_proc = total_pairs // size
    remaining_pairs = total_pairs % size

    if rank < remaining_pairs:
        pairs_for_this_proc = pairs_per_proc + 1
        start_pair = rank * (pairs_per_proc + 1)
    else:
        pairs_for_this_proc = pairs_per_proc
        start_pair = remaining_pairs * (pairs_per_proc + 1) + (rank -
remaining_pairs) * pairs_per_proc

```

```

all_pairs = []
for i in range(n):
    if i != removed_cluster_idx:
        for j in range(i + 1, n):
            if j != removed_cluster_idx:
                adj_i = i if i < removed_cluster_idx else i - 1
                adj_j = j if j < removed_cluster_idx else j - 1
                all_pairs.append((adj_i, adj_j, i, j))

end_pair = start_pair + pairs_for_this_proc
pairs_for_this_rank = all_pairs[start_pair:min(end_pair,
len(all_pairs))]

local_min_distance = float('inf')
local_min_pair = (-1, -1)

for adj_i, adj_j, orig_i, orig_j in pairs_for_this_rank:
    cluster_i = cluster_aktif[orig_i]
    cluster_j = cluster_aktif[orig_j]
    dist = matriks_jarak[cluster_i.id][cluster_j.id]
    if dist < local_min_distance:
        local_min_distance = dist
        local_min_pair = (adj_i, adj_j)

all_results = comm.gather((local_min_distance, local_min_pair),
root=0)

global_min_distance = float('inf')
global_min_pair = (-1, -1)

if rank == 0:
    for dist, pair in all_results:
        if dist < global_min_distance:
            global_min_distance = dist
            global_min_pair = pair

    if global_min_pair[0] != -1:
        print(f"Jarak minimum untuk iterasi berikutnya:
{global_min_distance:.4f}")

    global_result = comm.bcast((global_min_distance, global_min_pair),
root=0)
    global_min_distance, global_min_pair = global_result

return matriks_jarak, global_min_distance, global_min_pair

```

Script tersebut digunakan dalam proses *agglomerative hierarchical clustering* paralel untuk memperbarui matriks jarak setelah dua cluster digabung, lalu menentukan pasangan *cluster* berikutnya yang memiliki jarak terdekat. *Script* tersebut membagi tugas pembaruan jarak antar *cluster* secara merata ke setiap proses MPI berdasarkan indeks *cluster* yang aktif, lalu menghitung jarak baru antara *cluster* hasil gabungan dan *cluster* lainnya menggunakan metode *average linkage*. Setiap proses mengirimkan hasil pembaruan lokal ke proses utama (rank 0), yang kemudian menyatukan hasil tersebut ke dalam matriks jarak global. Setelah itu, *script* tersebut membagi total kemungkinan pasangan *cluster* baru ke setiap proses untuk

mencari jarak minimum berikutnya. Proses utama akan memilih pasangan dengan jarak terkecil sebagai kandidat penggabungan di iterasi selanjutnya.

```
def print_distance_matrix(matriks_jarak, cluster_aktif, iteration):
    n = len(cluster_aktif)

    print(f"\n=== MATRIX JARAK ITERASI {iteration} ===")
    print(f"Jumlah cluster aktif: {n}")
    print("Matriks jarak antar cluster (Average Linkage):")

    print("Cluster", end="")
    for j in range(n):
        print(f"{cluster_aktif[j].id+1:>8}", end="")
    print()

    for i in range(n):
        cluster_i = cluster_aktif[i]
        print(f"{cluster_i.id+1:>7}", end="")
        for j in range(n):
            cluster_j = cluster_aktif[j]
            if cluster_i.id == cluster_j.id:
                print(f"'0.00':>8)", end="")
            else:
                print(f"{matriks_jarak[cluster_i.id][cluster_j.id]:>8.2
f}", end="")
        print()

    print("\nUkuran cluster:")
    for cluster in cluster_aktif:
        print(f"Cluster {cluster.id+1}: {cluster.size} anggota")
    print("=" * 50)
```

Script tersebut digunakan untuk menampilkan isi matriks jarak antar *cluster* yang sedang aktif pada setiap iterasi proses *hierarchical clustering*. *Script* tersebut mencetak matriks jarak dalam format tabel, dengan label baris dan kolom yang menunjukkan ID masing-masing *cluster*. Setiap elemen dalam matriks menampilkan jarak rata-rata (*average linkage*) antar dua *cluster*, dengan nilai diagonal diatur menjadi 0.00 karena jarak sebuah *cluster* terhadap dirinya sendiri selalu nol. Selain itu, *script* tersebut juga mencetak jumlah anggota (*size*) dari setiap *cluster* aktif untuk memberikan informasi tambahan mengenai struktur *cluster* saat ini.

```
def visualize_distance_matrix(matriks_jarak, cluster_aktif, iteration,
save_file=True):
    n = len(cluster_aktif)

    if n <= 1:
        return

    plt.figure(figsize=(10, 8))

    indices = [cluster.id for cluster in cluster_aktif]
    sub_matrix = matriks_jarak[np.ix_(indices, indices)]

    cluster_labels = [f"C{cluster.id+1}\n({cluster.size})" for cluster
in cluster_aktif]
```

```

sns.heatmap(sub_matrix,
            annot=True,
            fmt='.2f',
            cmap='viridis',
            xticklabels=cluster_labels,
            yticklabels=cluster_labels,
            cbar_kws={'label': 'Jarak (Average Linkage)'})

plt.title(f'Matrix Jarak Antar Cluster - Iterasi {iteration}\n(Angka
dalam kurung = ukuran cluster)')
plt.xlabel('Cluster')
plt.ylabel('Cluster')

plt.tight_layout()

if save_file:
    if isinstance(iteration, int):
        filename = f'distance_matrix_iteration_{iteration:03d}.png'
    else:
        filename = f'distance_matrix_iteration_{iteration}.png'
    plt.savefig(filename, dpi=300, bbox_inches='tight')
    plt.show()

```

Script tersebut digunakan untuk memvisualisasikan matriks jarak antar *cluster* dalam bentuk *heatmap* menggunakan *library* “matplotlib” dan “seaborn”, yang sangat berguna untuk memahami hubungan antar *cluster* selama proses *hierarchical clustering*. *Script* tersebut pertama-tama menyusun submatriks dari *cluster* aktif berdasarkan ID mereka, lalu memetakan nilai jarak dalam bentuk warna menggunakan palet *viridis*. Label *cluster* ditampilkan dalam format “C[id]\n(size)” untuk menunjukkan identitas dan jumlah anggota masing-masing *cluster*. Visualisasi ini memudahkan pengguna dalam mengidentifikasi *cluster-cluster* yang saling berdekatan (jarak rendah) atau berjauhan (jarak tinggi).

```

def create_3d_scatter_plot(customers, clusters, title="Hasil Clustering
3D"):
    if len(customers) == 0:
        return

    fig = plt.figure(figsize=(12, 9))
    ax = fig.add_subplot(111, projection='3d')

    colors = ['red', 'blue', 'green', 'orange', 'purple', 'brown',
'pink', 'gray', 'olive', 'cyan']

    for i, cluster in enumerate(clusters):
        age = [customers[mid].umur for mid in cluster.anggota]
        income = [customers[mid].pendapatan for mid in cluster.anggota]
        spending = [customers[mid].pengeluaran for mid in
cluster.anggota]

        ax.scatter(age, income, spending,
                  c=colors[i % len(colors)],
                  label=f'Cluster {cluster.id+1}',
                  alpha=0.7,

```

```

s=60)

ax.set_xlabel('Umur')
ax.set_ylabel('Pendapatan tahunan (k$)')
ax.set_zlabel('skor pengeluaran (1-100)')
ax.set_title(title)
ax.legend()

ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('clustering_3d_results.png',      dpi=300,
bbox_inches='tight')
plt.show()

```

Script tersebut digunakan untuk membuat visualisasi hasil *clustering* dalam bentuk grafik sebar 3 dimensi berdasarkan tiga atribut pelanggan: “umur”, “pendapatan tahunan”, dan “skor pengeluaran”. *Script* tersebut memanfaatkan *library* matplotlib dengan proyeksi 3D untuk memetakan setiap *cluster* ke dalam ruang tiga dimensi, di mana setiap *cluster* diberi warna berbeda dari daftar warna yang disediakan. Setiap titik dalam plot mewakili seorang pelanggan yang tergabung dalam *cluster* tertentu, dan label *cluster* ditampilkan di legenda untuk membedakan antar kelompok. Sumbu-sumbu diberi label sesuai atribut, dan plot disimpan sebagai gambar PNG agar dapat dianalisis atau didokumentasikan.

```

def main():
    mulai = time.time()

    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    customers = []
    banyak_customers = 0
    cluster_tujuan = 5
    hasil_clustering = HasilClustering()

    if rank == 0:
        print("=== HIERARCHICAL CLUSTERING ===")
        print(f"Jumlah proses MPI: {size}")
        np.random.seed(42)
        banyak_customers = 500

        age = np.random.randint(18, 71, banyak_customers)
        income = np.random.randint(1000000, 20000001, banyak_customers)
        spending_scores = np.random.randint(1, 101, banyak_customers)

        print(f"Menghasilkan {banyak_customers} data pelanggan secara
random")
        print("Rentang data:")
        print("- Usia: 18-70 tahun")
        print("- Pendapatan Tahunan: 15k-137k")
        print("- Spending Score: 1-100")

```

```

for i in range(banyak_customers):
    customer = Customer(i, age[i], income[i], spending_scores[i])
    customers.append(customer)

normalisasi_data(customers)

age_dist = [0, 0, 0]
income_dist = [0, 0]
spending_dist = [0, 0]

for customer in customers:
    age_dist[int(customer.fitur[0]) - 1] += 1
    income_dist[int(customer.fitur[1]) - 1] += 1
    spending_dist[int(customer.fitur[2]) - 1] += 1

print(f"\nDistribusi setelah normalisasi:")
print(f"Usia          - Remaja(1): {age_dist[0]}, Dewasa(2): {age_dist[1]}, Lansia(3): {age_dist[2]}")
print(f"Pendapatan    - Rendah(1): {income_dist[0]}, Tinggi(2): {income_dist[1]}")
print(f"Spending      - Rendah(1): {spending_dist[0]}, Tinggi(2): {spending_dist[1]}")

banyak_customers = comm.bcast(banyak_customers, root=0)
customers = comm.bcast(customers, root=0)

data_prep_time = time.time()

```

Script tersebut merupakan fungsi “main()” untuk menjalankan program *Hierarchical Clustering* berbasis paralel menggunakan MPI. *Script* tersebut dimulai dengan mencatat waktu awal eksekusi dan inisialisasi komunikasi MPI, termasuk mengambil rank dan size . Proses utama “(rank == 0)” bertanggung jawab membuat dataset pelanggan secara acak sebanyak 500 data dengan atribut umur, pendapatan, dan skor pengeluaran. Dataset kemudian dinormalisasi dan dikategorikan ke dalam kelompok seperti remaja/dewasa/lansia untuk usia, serta rendah/tinggi untuk pendapatan dan pengeluaran, yang distribusinya ditampilkan ke layar. Setelah data disiapkan, *script* tersebut melakukan *broadcast* data pelanggan dan jumlah total pelanggan ke seluruh proses MPI, agar setiap proses memiliki data yang sama untuk digunakan dalam perhitungan paralel selanjutnya.

```

if rank == 0:
    print(f"\nMembuat matriks jarak dengan grid-based distribution...")
    print(f"Menghitung jarak Euclidean untuk semua {banyak_customers*(banyak_customers-1)//2} pasangan data...")

    matriks_jarak, next_min_distance, next_min_pair =
    buat_matriks_jarak_awal_blok(customers, comm, rank, size)

    matrix_build_time = time.time()

    if rank == 0:
        print(f"Matriks jarak berhasil dibangun dalam {matrix_build_time - data_prep_time:.4f} detik")
        print(f"Minimum distance : {next_min_distance:.4f}")

```

```

        print(f"Minimum pair : Customer {next_min_pair[0]+1} dan Customer
{next_min_pair[1]+1}")

    cluster_aktif = []
    if rank == 0:
        for i in range(banyak_customers):
            cluster = Cluster(i, [i])
            cluster.size = 1
            cluster_aktif.append(cluster)

    print(f"\nMemulai clustering dari {len(cluster_aktif)} cluster
ke {cluster_tujuan} cluster")
    print("Proses clustering dengan Average Linkage dimulai...")

    cluster_aktif = comm.bcast(cluster_aktif, root=0)

    iteration = 0
    clustering_start_time = time.time()

    while len(cluster_aktif) > cluster_tujuan:
        iteration += 1

        if rank == 0:
            print(f"\n--- Iterasi {iteration} ---")
            print(f"Jumlah cluster aktif: {len(cluster_aktif)}")

            if iteration == 1 or len(cluster_aktif) <= cluster_tujuan +
2 or iteration % 500 == 0:
                visualize_distance_matrix(matriks_jarak, cluster_aktif,
iteration)

            if iteration == 1:
                jarak_terdekat = next_min_distance
                if rank == 0:
                    cluster_i_idx = cluster_j_idx = -1
                    for idx, cluster in enumerate(cluster_aktif):
                        if next_min_pair[0] in cluster.anggota:
                            cluster_i_idx = idx
                        if next_min_pair[1] in cluster.anggota:
                            cluster_j_idx = idx

                    if cluster_i_idx == -1 or cluster_j_idx == -1:
                        print("Error: Tidak dapat menemukan cluster dari
initial minimum pair!")
                        break
                    else:
                        cluster_i_idx = cluster_j_idx = None

                info_gabungan = comm.bcast((cluster_i_idx, cluster_j_idx,
jarak_terdekat), root=0)
                cluster_i_idx, cluster_j_idx, jarak_terdekat = info_gabungan

```

Script tersebut melanjutkan proses *hierarchical clustering* setelah tahap persiapan data, dengan membangun matriks jarak antar pelanggan secara paralel melalui fungsi “*buat_matriks_jarak_awal_blok*”, lalu memulai proses penggabungan *cluster* berdasarkan *average linkage*. Jika proses berada di rank 0, maka ditampilkan informasi awal seperti jumlah pasangan jarak yang akan dihitung dan waktu yang dibutuhkan untuk

membentuk matriks. Setelah itu, seluruh data *cluster* awal dibroadcast ke semua proses. Iterasi penggabungan *cluster* dilakukan di dalam loop while, selama jumlah *cluster* aktif masih lebih banyak dari “`cluster_tujuan`”. Visualisasi matriks jarak ditampilkan pada iterasi pertama, terakhir, atau kelipatan tertentu. Untuk iterasi pertama, jarak minimum dan pasangan *cluster* awal yang ditemukan saat membentuk matriks digunakan sebagai dasar penggabungan awal. *Script* tersebut kemudian mencari indeks *cluster* aktif dari pasangan pelanggan terdekat tersebut, lalu membroadcast informasi penggabungan ke seluruh proses untuk memastikan setiap proses memiliki pemahaman yang konsisten sebelum proses update dan penggabungan *cluster* berikutnya dilakukan.

```

else:
    jarak_terdekat = next_min_distance
    cluster_i_idx, cluster_j_idx = next_min_pair

    if cluster_i_idx is None or cluster_i_idx == -1:
        if rank == 0:
            print("Clustering dihentikan: tidak ada pasangan cluster
yang valid.")
            break

    if rank == 0:
        cluster_i = cluster_aktif[cluster_i_idx]
        cluster_j = cluster_aktif[cluster_j_idx]
        print(f"Menggabungkan Cluster {cluster_i.id+1} (size:
{cluster_i.size}) "
              f"dengan Cluster {cluster_j.id+1} (size:
{cluster_j.size})")
        print(f"Jarak: {jarak_terdekat:.4f}")

        hasil_clustering.merge_history.append((cluster_i.id,
cluster_j.id))
        hasil_clustering.jarak.append(jarak_terdekat)
        hasil_clustering.clusters_at_each_step.append(len(cluster_a
ktif))

        cluster_i = cluster_aktif[cluster_i_idx]
        cluster_j = cluster_aktif[cluster_j_idx]

        cluster_i.anggota.extend(cluster_j.anggota)
        cluster_i.size = len(cluster_i.anggota)

        matriks_jarak, next_min_distance, next_min_pair =
update_distance_matrix_and_find_minimum(
            matriks_jarak, cluster_i_idx, cluster_j_idx,
            cluster_aktif, customers, comm, rank, size
        )

        cluster_aktif.pop(cluster_j_idx)

        if rank == 0:
            progress = ((banyak_customers - len(cluster_aktif)) /
(banyak_customers - cluster_tujuan)) * 100
            print(f"Progress: {progress:.1f}% (Cluster tersisa:
{len(cluster_aktif)})")

        clustering_end_time = time.time()

```

Script tersebut melanjutkan proses utama *hierarchical clustering* setelah iterasi pertama, dengan menggunakan pasangan *cluster* dengan jarak terdekat yang ditemukan untuk iterasi berikutnya. Jika proses bukan rank 0, maka informasi pasangan *cluster* terdekat dan jaraknya akan digunakan langsung. *Script* tersebut memastikan bahwa pasangan *cluster* valid sebelum melanjutkan, jika tidak, proses *clustering* dihentikan. Jika proses berada pada rank 0, maka ditampilkan informasi penggabungan dua *cluster*: ID *cluster*, ukuran masing-masing, dan jarak antar *cluster*. Informasi penggabungan ini disimpan dalam objek “hasil_clustering” untuk dokumentasi hasil *clustering*. Kemudian *cluster j* digabungkan

ke *cluster* *i*, dan ukuran *cluster* *i* diperbarui. Setelah penggabungan, fungsi “update_distance_matrix_and_find_minimum” dipanggil untuk memperbarui matriks jarak dan menentukan pasangan *cluster* terdekat berikutnya secara paralel. *Cluster* *j* dihapus dari daftar *cluster* aktif karena telah digabung. Setelah setiap iterasi, jika proses adalah rank 0, maka progres *clustering* ditampilkan sebagai persentase dari *cluster* yang telah digabung. Iterasi berlanjut hingga jumlah *cluster* aktif mencapai target *cluster_tujuan*, lalu waktu akhir proses *clustering* dicatat.

```

if rank == 0:
    print(f"\nClustering selesai!")
    print(f"="*70)

    print(f"\nMATRIX JARAK FINAL:")
    print_distance_matrix(matriks_jarak, cluster_aktif, "Final")
    visualize_distance_matrix(matriks_jarak, cluster_aktif, "Final")

    print(f"\nHASIL CLUSTERING AKHIR:")
    print(f"Jumlah cluster final: {len(cluster_aktif)}")
    print(f"Target cluster: {cluster_tujuan}")
    print(f"Metode: Average Linkage (Optimized - Combined
Operations)")
    print(f"Jumlah iterasi: {iteration}")

    total_customers_in_clusters = sum(cluster.size for cluster in
cluster_aktif)
    print(f"Total pelanggan dalam cluster:
{total_customers_in_clusters}")

    for i, cluster in enumerate(cluster_aktif):
        print(f"\nCluster {cluster.id+1} (ukuran: {cluster.size}
pelanggan, {cluster.size/total_customers_in_clusters*100:.1f}%):")

        umur_awal = [customers[mid].umur for mid in cluster.anggota]
        pendapatan_awal = [customers[mid].pendapatan for mid in
cluster.anggota]
        pengeluaran_awal = [customers[mid].pengeluaran for mid in
cluster.anggota]

        print(f" Karakteristik:")
        print(f"      Usia: {min(umur_awal)}-{max(umur_awal)} tahun
(rata-rata: {np.mean(umur_awal):.1f})")
        print(f"      Pendapatan: {min(pendapatan_awal)}-
{max(pendapatan_awal)}k (rata-rata: {np.mean(pendapatan_awal):.1f}k)")
        print(f"      Spending: {min(pengeluaran_awal)}-
{max(pengeluaran_awal)} (rata-rata: {np.mean(pengeluaran_awal):.1f})")

        if cluster.size <= 10:
            print(f" Anggota: {[mid+1 for mid in cluster.anggota]}")

    print(f"\nINTERPRETASI CLUSTER:")
    for i, cluster in enumerate(cluster_aktif):
        umur_awal = [customers[mid].umur for mid in cluster.anggota]
        pendapatan_awal = [customers[mid].pendapatan for mid in
cluster.anggota]
        pengeluaran_awal = [customers[mid].pengeluaran for mid in
cluster.anggota]

```



```

rata_rata_umur = np.mean(umur_awal)
avg_income = np.mean(pendapatan_awal)
rata_rata_pengeluaran = np.mean(pengeluaran_awal)

age_cat = "Muda" if rata_rata_umur < 30 else "Dewasa" if
rata_rata_umur < 50 else "Lansia"
income_cat = "Rendah" if avg_income < 60 else "Menengah" if
avg_income < 100 else "Tinggi"
spending_cat = "Rendah" if rata_rata_pengeluaran < 40 else
"Sedang" if rata_rata_pengeluaran < 70 else "Tinggi"

print(f"Cluster {i+1}: Pelanggan {age_cat} dengan Pendapatan
{income_cat} dan Spending {spending_cat}")

if avg_income > 80 and rata_rata_pengeluaran > 60:
    strategy = "Produk premium, program loyalitas pelanggan"
elif avg_income < 50 and rata_rata_pengeluaran < 40:
    strategy = "Produk ekonomis, diskon menarik"
elif rata_rata_umur < 30:
    strategy = "Produk kekinian, pemasaran lewat media sosial"
elif rata_rata_umur > 50:
    strategy = "Produk berkualitas, pemasaran tradisional"
else:
    strategy = "Pendekatan seimbang, promosi yang ditargetkan"

print(f"  Strategi: {strategy}")

print(f"\nMembuat visualisasi 3D...")

visualization_start = time.time()

create_3d_scatter_plot(customers, cluster_aktif, "Pengelompokan
Customer")

visualization_time = time.time()

total_time = time.time() - mulai
print(f"\nTOTAL WAKTU EKSEKUSI: {total_time:.4f} detik")
print(f"*70)

if __name__ == "__main__":
    main()

```

Script tersebut merupakan bagian akhir dari fungsi “main()” yang dijalankan oleh proses utama “(rank == 0)” setelah proses *hierarchical clustering* selesai. *Script* ini mencetak dan menyimpulkan hasil akhir dari proses *clustering*. Pertama, matriks jarak akhir ditampilkan. Kemudian, informasi penting seperti jumlah *cluster* akhir, target *cluster*, jumlah iterasi, dan jumlah pelanggan dalam *cluster* dicetak sebagai ringkasan. Untuk setiap *cluster* yang terbentuk, *script* menampilkan karakteristiknya berdasarkan rentang dan rata-rata atribut umur, pendapatan, dan pengeluaran, serta menampilkan anggota *cluster* jika jumlahnya kecil (≤ 10). Selanjutnya, dilakukan interpretasi *cluster* berdasarkan kategori usia, tingkat pendapatan dan pola pengeluaran, lalu diberi rekomendasi strategi pemasaran yang sesuai untuk masing-masing kelompok.

B. HASIL PERCOBAAN

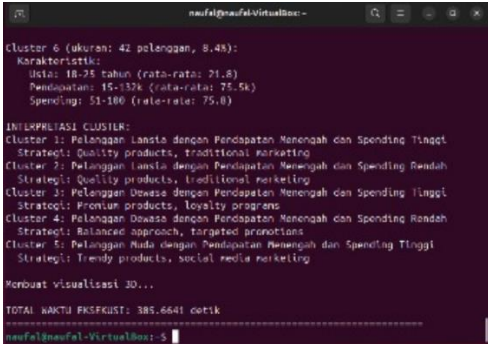
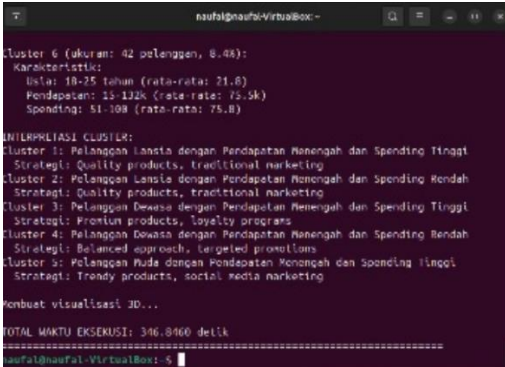
1. 100 Data

1 Komputer	3 Komputer
<pre> INTERPRETASI CLUSTER: Cluster 1: Pelanggan Dewasa dengan Pendapatan Rendah dan Spending Rendah Strategi: Budget-friendly options, discounts Cluster 2: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Sedang Strategi: Quality products, traditional marketing Cluster 3: Pelanggan Dewasa dengan Pendapatan Rendah dan Spending Tinggi Strategi: Balanced approach, targeted promotions Cluster 4: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 5: Pelanggan Muda dengan Pendapatan Rendah dan Spending Rendah Strategi: Budget-friendly options, discounts Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 43.9473 detik ===== naufal@naufal-VirtualBox: \$ </pre>	<pre> Cluster 62 (ukuran: 2 pelanggan, 2.0%): Karakteristik: Usia: 24-25 tahun (rata-rata: 24.5) Pendapatan: 23-73k (rata-rata: 48.8k) Spending: 33-38 (rata-rata: 35.5) Anggota: [62, 98] INTERPRETASI CLUSTER: Cluster 1: Pelanggan Dewasa dengan Pendapatan Rendah dan Spending Rendah Strategi: Budget-friendly options, discounts Cluster 2: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Sedang Strategi: Quality products, traditional marketing Cluster 3: Pelanggan Dewasa dengan Pendapatan Rendah dan Spending Tinggi Strategi: Balanced approach, targeted promotions Cluster 4: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 5: Pelanggan Muda dengan Pendapatan Rendah dan Spending Rendah Strategi: Budget-friendly options, discounts Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 24.1774 detik ===== naufal@naufal-VirtualBox: \$ </pre>
2 Komputer	4 Komputer
<pre> Cluster 62 (ukuran: 2 pelanggan, 2.0%): Karakteristik: Usia: 24-25 tahun (rata-rata: 24.5) Pendapatan: 23-73k (rata-rata: 48.8k) Spending: 33-38 (rata-rata: 35.5) Anggota: [62, 98] INTERPRETASI CLUSTER: Cluster 1: Pelanggan Dewasa dengan Pendapatan Rendah dan Spending Rendah Strategi: Budget-friendly options, discounts Cluster 2: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Sedang Strategi: Quality products, traditional marketing Cluster 3: Pelanggan Dewasa dengan Pendapatan Rendah dan Spending Tinggi Strategi: Balanced approach, targeted promotions Cluster 4: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 5: Pelanggan Muda dengan Pendapatan Rendah dan Spending Rendah Strategi: Budget-friendly options, discounts Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 25.4632 detik ===== naufal@naufal-VirtualBox: \$ </pre>	<pre> Cluster 62 (ukuran: 2 pelanggan, 2.0%): Karakteristik: Usia: 24-25 tahun (rata-rata: 24.5) Pendapatan: 23-73k (rata-rata: 48.8k) Spending: 33-38 (rata-rata: 35.5) Anggota: [62, 88] INTERPRETASI CLUSTER: Cluster 1: Pelanggan Dewasa dengan Pendapatan Rendah dan Spending Rendah Strategi: Budget-friendly options, discounts Cluster 2: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Sedang Strategi: Quality products, traditional marketing Cluster 3: Pelanggan Dewasa dengan Pendapatan Rendah dan Spending Tinggi Strategi: Balanced approach, targeted promotions Cluster 4: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 5: Pelanggan Muda dengan Pendapatan Rendah dan Spending Rendah Strategi: Budget-friendly options, discounts Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 18.3827 detik ===== naufal@naufal-VirtualBox: \$ </pre>

2. 200 Data

1 Komputer	3 Komputer
<pre> Spending: 4-100 (rata-rata: 44.2) INTERPRETASI CLUSTER: Cluster 1: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Tinggi Strategi: Quality products, traditional marketing Cluster 2: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 3: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Rendah Strategi: Quality products, traditional marketing Cluster 4: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 5: Pelanggan Muda dengan Pendapatan Tinggi dan Spending Sedang Strategi: Trendy products, social media marketing Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 92.8141 detik ===== PS D:\Semester 4\Pemrosesan Paralel\Project: </pre>	<pre> Cluster 21 (ukuran: 14 pelanggan, 7.0%): Karakteristik: Usia: 18-25 tahun (rata-rata: 20.1) Pendapatan: 82-137k (rata-rata: 104.6k) Spending: 4-100 (rata-rata: 44.2) INTERPRETASI CLUSTER: Cluster 1: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Tinggi Strategi: Quality products, traditional marketing Cluster 2: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 3: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Rendah Strategi: Quality products, traditional marketing Cluster 4: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 5: Pelanggan Muda dengan Pendapatan Tinggi dan Spending Sedang Strategi: Trendy products, social media marketing Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 49.6999 detik ===== naufal@naufal-VirtualBox: \$ </pre>
2 Komputer	4 Komputer
<pre> Cluster 21 (ukuran: 14 pelanggan, 7.0%): Karakteristik: Usia: 18-25 tahun (rata-rata: 20.1) Pendapatan: 82-137k (rata-rata: 104.6k) Spending: 4-100 (rata-rata: 44.2) INTERPRETASI CLUSTER: Cluster 1: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Tinggi Strategi: Quality products, traditional marketing Cluster 2: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 3: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Rendah Strategi: Quality products, traditional marketing Cluster 4: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 5: Pelanggan Muda dengan Pendapatan Tinggi dan Spending Sedang Strategi: Trendy products, social media marketing Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 59.8866 detik ===== naufal@naufal-VirtualBox: \$ </pre>	<pre> Cluster 21 (ukuran: 14 pelanggan, 7.0%): Karakteristik: Usia: 18-25 tahun (rata-rata: 20.1) Pendapatan: 82-137k (rata-rata: 104.6k) Spending: 4-100 (rata-rata: 44.2) INTERPRETASI CLUSTER: Cluster 1: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Tinggi Strategi: Quality products, traditional marketing Cluster 2: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 3: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Rendah Strategi: Quality products, traditional marketing Cluster 4: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 5: Pelanggan Muda dengan Pendapatan Tinggi dan Spending Sedang Strategi: Trendy products, social media marketing Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 48.9834 detik ===== naufal@naufal-VirtualBox: \$ </pre>

5. 500 Data

1 Komputer	3 Komputer
Pendapatan: 15-132k (rata-rata: 75.5k) Spending: 51-100 (rata-rata: 75.8) INTERPRETASI CLUSTER: Cluster 1: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Tinggi Strategi: Quality products, traditional marketing Cluster 2: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Rendah Strategi: Quality products, traditional marketing Cluster 3: Pelanggan Dewasa dengan Pendapatan Menengah dan Spending Tinggi Strategi: Premium products, loyalty programs Cluster 4: Pelanggan Dewasa dengan Pendapatan Menengah dan Spending Rendah Strategi: Balanced approach, targeted promotions Cluster 5: Pelanggan Muda dengan Pendapatan Menengah dan Spending Tinggi Strategi: Trendy products, social media marketing Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 666.6255 detik =====	
2 Komputer	4 Komputer
	

6. 600 Data

1 Komputer	3 Komputer
Cluster 18 (ukuran: 59 pelanggan, 9.8%): Karakteristik: Usia: 18-25 tahun (rata-rata: 20.9) Pendapatan: 17-136k (rata-rata: 95.5k) Spending: 3-100 (rata-rata: 59.4) INTERPRETASI CLUSTER: Cluster 1: Pelanggan Lansia dengan Pendapatan Tinggi dan Spending Rendah Strategi: Quality products, traditional marketing Cluster 2: Pelanggan Dewasa dengan Pendapatan Menengah dan Spending Tinggi Strategi: Balanced approach, targeted promotions Cluster 3: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Sedang Strategi: Quality products, traditional marketing Cluster 4: Pelanggan Dewasa dengan Pendapatan Rendah dan Spending Rendah Strategi: Budget-friendly options, discounts Cluster 5: Pelanggan Muda dengan Pendapatan Menengah dan Spending Sedang Strategi: Trendy products, social media marketing Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 935.6448 detik	
2 Komputer	4 Komputer
	

7. 700 Data

1 Komputer	3 Komputer
<pre> INTERPRETASI CLUSTER: Cluster 1: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Sedang Strategi: Quality products, traditional marketing Cluster 2: Pelanggan Dewasa dengan Pendapatan Rendah dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 3: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Rendah Strategi: Balanced approach, targeted promotions Cluster 4: Pelanggan Muda dengan Pendapatan Menengah dan Spending Tinggi Strategi: Trendy products, social media marketing Cluster 5: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Tinggi Strategi: Premium products, loyalty programs Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 1010.2737 detik ===== </pre>	<pre> Cluster 20 (ukuran: 144 pelanggan, 20.6%): Karakteristik: Usia: 26-70 tahun (rata-rata: 49.5) Pendapatan: 77-137k (rata-rata: 105.3k) Spending: 51-100 (rata-rata: 75.8) INTERPRETASI CLUSTER: Cluster 1: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Sedang Strategi: Quality products, traditional marketing Cluster 2: Pelanggan Dewasa dengan Pendapatan Rendah dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 3: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Rendah Strategi: Balanced approach, targeted promotions Cluster 4: Pelanggan Muda dengan Pendapatan Menengah dan Spending Tinggi Strategi: Trendy products, social media marketing Cluster 5: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Tinggi Strategi: Premium products, loyalty programs Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 610.2354 detik ===== </pre>
2 Komputer	4 Komputer
<pre> Cluster 20 (ukuran: 144 pelanggan, 20.6%): Karakteristik: Usia: 26-70 tahun (rata-rata: 49.5) Pendapatan: 77-137k (rata-rata: 105.3k) Spending: 51-100 (rata-rata: 75.8) INTERPRETASI CLUSTER: Cluster 1: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Sedang Strategi: Quality products, traditional marketing Cluster 2: Pelanggan Dewasa dengan Pendapatan Rendah dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 3: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Rendah Strategi: Balanced approach, targeted promotions Cluster 4: Pelanggan Muda dengan Pendapatan Menengah dan Spending Tinggi Strategi: Trendy products, social media marketing Cluster 5: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Tinggi Strategi: Premium products, loyalty programs Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 636.2570 detik ===== </pre>	<pre> Cluster 20 (ukuran: 144 pelanggan, 20.6%): Karakteristik: Usia: 26-70 tahun (rata-rata: 49.5) Pendapatan: 77-137k (rata-rata: 105.3k) Spending: 51-100 (rata-rata: 75.8) INTERPRETASI CLUSTER: Cluster 1: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Sedang Strategi: Quality products, traditional marketing Cluster 2: Pelanggan Dewasa dengan Pendapatan Rendah dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 3: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Rendah Strategi: Balanced approach, targeted promotions Cluster 4: Pelanggan Muda dengan Pendapatan Menengah dan Spending Tinggi Strategi: Trendy products, social media marketing Cluster 5: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Tinggi Strategi: Premium products, loyalty programs Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 603.5332 detik ===== </pre>

8. 500 data

8 N	12 N
<pre> INTERPRETASI CLUSTER: Cluster 1: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Tinggi Strategi: Quality products, traditional marketing Cluster 2: Pelanggan Lansia dengan Pendapatan Tinggi dan Spending Sedang Strategi: Quality products, traditional marketing Cluster 3: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Sedang Strategi: Quality products, traditional marketing Cluster 4: Pelanggan Dewasa dengan Pendapatan Rendah dan Spending Tinggi Strategi: Basic products, broad reach marketing Cluster 5: Pelanggan Muda dengan Pendapatan Menengah dan Spending Tinggi Strategi: Trendy products, social media marketing Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 334.172 detik naufal@naufal-VirtualBox: ~/Documents \$ </pre>	<pre> INTERPRETASI CLUSTER: Cluster 1: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Rendah Strategi: Basic products, broad reach marketing Cluster 2: Pelanggan Muda dengan Pendapatan Tinggi dan Spending Tinggi Strategi: Trendy products, social media marketing Cluster 3: Pelanggan Muda dengan Pendapatan Tinggi dan Spending Tinggi Strategi: Trendy products, social media marketing Cluster 4: Pelanggan Muda dengan Pendapatan Tinggi dan Spending Rendah Strategi: Trendy products, social media marketing Cluster 5: Pelanggan Muda dengan Pendapatan Menengah dan Spending Tinggi Strategi: Trendy products, social media marketing Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 274.982 detik naufal@naufal-VirtualBox: ~/Documents \$ </pre>
16 N	20 N
<pre> INTERPRETASI CLUSTER: Cluster 1: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Tinggi Strategi: Premium products, exclusive membership Cluster 2: Pelanggan Lansia dengan Pendapatan Rendah dan Spending Tinggi Strategi: Quality products, traditional marketing Cluster 3: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Sedang Strategi: Quality products, traditional marketing Cluster 4: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Tinggi Strategi: Balanced approach, targeted promotions Cluster 5: Pelanggan Muda dengan Pendapatan Tinggi dan Spending Sedang Strategi: Trendy products, social media marketing Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 254.662 detik naufal@naufal-VirtualBox: ~/Documents \$ </pre>	<pre> INTERPRETASI CLUSTER: Cluster 1: Pelanggan Dewasa dengan Pendapatan Tinggi dan Spending Sedang Strategi: Premium products, exclusive membership Cluster 2: Pelanggan Dewasa dengan Pendapatan Menengah dan Spending Sedang Strategi: Balanced approach, targeted promotions Cluster 3: Pelanggan Lansia dengan Pendapatan Menengah dan Spending Tinggi Strategi: Quality products, traditional marketing Cluster 4: Pelanggan Dewasa dengan Pendapatan Menengah dan Spending Rendah Strategi: Trendy products, social media marketing Cluster 5: Pelanggan Muda dengan Pendapatan Rendah dan Spending Rendah Strategi: Trendy products, social media marketing Membuat visualisasi 3D... TOTAL WAKTU EKSEKUSI: 224.276 detik naufal@naufal-VirtualBox: ~/Documents \$ </pre>